M02-L02 · Organización de specs

Alberto Basalo

TESTING INTELIGENTE

Generar pruebas con skills

> *La spec contiene historias. Cada historia contiene requisitos. La prueba repite ese orden.*

CONNECTIONS

- En el análisis, ¿qué contiene a qué: la **spec**, la historia o el requisito?
- ¿Dónde está la prueba de Playwright de cada **nivel**?

CONCEPTS · SCAFFOLDING EN EL ANÁLISIS

Spec

Una funcionalidad completa, en su carpeta.

- Patrón: `specs/<id>-<slug>/spec.md` - Ejemplo: `specs/S0003-schedule-launch/spec.md`

Historia

Lo que pide un usuario, en un `.feature`.

- Patrón: `US<n>-<slug>.feature` - Ejemplo: `US1-schedule-launch.feature`

CONCEPTS · SCAFFOLDING EN PLAYWRIGHT

Spec

Una carpeta en `e2e/tests` con el mismo patrón que en `specs/` .

- Patrón: `e2e/tests/<id>-<slug>/` - Ejemplo: `e2e/tests/S0003-schedule-launch/`

Historia

Un archivo `.test.ts` con un `describe` . (alternativa aceptable al original **.spec.ts*)

El `describe` lleva la etiqueta de la spec y la de la historia. Sus `test` las heredan.

- Patrón: `US<n>-<slug>.test.ts` · `describe('[US<n>] …', { tag: ['@<id>', '@<id>-US<n>'] })`
- Ejemplo: `US1-schedule-launch.test.ts` · `@S0003` · `@S0003-US1`

CONCEPTS · REQUISITOS

Criterios en el análisis

Un criterio EARS de una sola historia. Sus escenarios lo citan; lo normal es uno.

➢ Patrón: `R<n> (US<n>)` - Ejemplo: `R3 (US2) – THE system SHALL do things`

Test de escenarios en Playwright

Un `describe(User Story)` en cada fichero de pruebas.

Un `test.(Requirement)` por cada escenario gherkin sacado de cada rqeurimiento

Tags de *Spec*, *UserStory* y *Requirement* en cada nivel.

CONCRETE PRACTICE

Revisar lo que hay

Pide el listado de la spec `S0003`: historias, requisitos y pruebas.

Al terminar L01, los archivos están sueltos. Nada indica a qué historia pertenece la prueba.

CONCRETE PRACTICE

Añadir la organización a la skill

Amplía `/implement-e2e-features` con carpetas, archivos y etiquetas.

Actualiza las rutas de `/define-spec` y `/define-e2e-features`. La próxima spec nace en su carpeta.

CONCRETE PRACTICE

Reorganizar lo ya generado

Es un cambio de una sola vez: basta un prompt.

El agente mueve los archivos y pone las etiquetas. Las comprobaciones no cambian.

CONCRETE PRACTICE

Generar los casos pendientes

Pide las pruebas de los escenarios sin cubrir. La skill aplica las reglas y la organización.

CONCRETE PRACTICE

Ejecutar una spec, una historia o un requisito

Desde `e2e/`:

```
# Toda la spec
npx playwright test --grep '@S0003'

# Una historia
npx playwright test --grep '@S0003-US1'

# Un requisito
npx playwright test --grep '@S0003-R1'
```

Antes de ejecutar, `--list` muestra qué pruebas selecciona cada comando.

Etiquetas

CONCLUSION

Tres niveles, dos sitios

Ves la spec, la historia y el requisito en el análisis y en la prueba de Playwright.

Buscar un requisito

Si cambia un requisito, ¿puedes señalar **su historia** y **su prueba**?

Siguiente paso

En L03 reunirás las interacciones repetidas en Page Objects.

Alberto Basalo